TA10 Actividades de aprendizaje profundo

Índice

1. Introducción
2. LeNet
3. Capas convolucionales
4. Pooling
5. ImageNet
6. AlexNet
7. Dropout
8. Inception
9. ResNet
10. Capas de normalización
11. Transformer
12. Vision Transformer

2 LeNet

Cuestión. LeNet es una red convolucional (CNN) pionera, introducida en 1998 por Yann LeCun. En relación con la misma, indica la opción incorrecta o escoge la última opción si las tres primeras son correctas:

1. Se diseñó para clasificar dígitos manuscritos, concretamente con el dataset MNIST.
2. Consta de dos capas convolucionales, dos de agrupación y tres lineales.
3. Tanto las capas convolucionales como las lineales, salvo la última, incluyen una función de activación sigmoide.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

3 Capas convolucionales

Problema: Halla la valid convolution de $\mathbf{X} = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$ con $\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

Solución: $\mathbf{Y} = \mathbf{W} \circledast \mathbf{X} = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix}$

In [ ]:
```
import numpy as np; import torch
W = torch.nn.Conv2d(1, 1, 2, bias=False)
W.weight.data = torch.tensor([[ [[1.,1.], [1.,1.]] ]], requires_grad=True)
X = torch.tensor([[ [[0,1,1], [0,1,0], [1,1,0]] ]], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[2., 3.],          [3., 2.]]]])
```

Problema: Halla la same convolution (zero-padding) de $\mathbf{X} = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$ con $\mathbf{W} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

Solución: $\mathbf{Y} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \circledast \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 3 \\ 4 & 5 & 4 \\ 3 & 3 & 2 \end{bmatrix}$

In [ ]:
```
W = torch.nn.Conv2d(1, 1, 2, padding='same', bias=False)
W.weight.data = torch.tensor([[ [[1.,1.,1.], [1.,1.,1.], [1.,1.,1.]] ]], requires_grad=True)
X = torch.tensor([ [[0,1,1], [0,1,0], [1,1,0]] ]], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[2., 3., 3.],          [4., 5., 4.],          [3., 3., 2.]]]])
```

Problema: Halla la strided convolution, con saltos de 2 en altura y anchura, de:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \quad \text{con} \quad \mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Solución: $\mathbf{Y} = \begin{bmatrix} 3 & 3 \\ 3 & 2 \end{bmatrix}$

In [ ]:
```
W = torch.nn.Conv2d(1, 1, 2, stride=2, padding=0, bias=False)
W.weight.data = torch.tensor([[ [[1,1], [1,1]] ]], requires_grad=True, dtype=torch.float)
X = torch.tensor([[ [[1,1,1,1], [1,0,0,1], [1,1,1,1], [1,0,0,0]] ]], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[3., 3.],          [3., 2.]]]])
```

Problema: Halla la valid convolution de:

$$\mathbf{X} = \left[\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix} \right] \quad \text{con} \quad \mathbf{W} = \left[\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \right]$$

Solución: $\mathbf{Y} = \mathbf{W} \circledast \mathbf{X} = \begin{bmatrix} 7 & 8 \\ 4 & 8 \end{bmatrix}$

In [ ]:
```
W = torch.nn.Conv2d(2, 1, 2, bias=False)
W.weight.data = torch.tensor([ [ [[1.,1.],[1.,1.]], [[2.,2.],[2.,2.]] ] ], requires_grad=True)
X = torch.tensor([ [ [[1,0,0],[1,1,1],[1,1,1]], [[1,1,1],[0,0,1],[0,0,1]] ] ], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[7., 8.],          [4., 8.]]]])
```

Problema: Halla la valid convolution de:

$$\mathbf{X} = \left[\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix} \right] \quad \text{con} \quad \mathbf{W} = \left[\left[\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \right], \left[\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \right] \right]$$

Solución: $\mathbf{Y} = \mathbf{W} \circledast \mathbf{X} = \left[\begin{bmatrix} 7 & 8 \\ 4 & 8 \end{bmatrix}, \begin{bmatrix} 5 & 5 \\ 4 & 6 \end{bmatrix} \right]$

In [ ]:
```
W = torch.nn.Conv2d(2, 2, 2, bias=False)
W.weight.data = torch.tensor([ [ [[1,1],[1,1]], [[2,2],[2,2]] ],
                               [ [[1,1],[1,1]], [[1,1],[1,1]] ] ], requires_grad=True, dtype=torch.float)
X = torch.tensor([ [ [[1,0,0],[1,1,1],[1,1,1]], [[1,1,1],[0,0,1],[0,0,1]] ] ], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[7., 8.],          [4., 8.]],         [[5., 5.],          [4., 6.]]]])
```

Problema: Halla la convolución 1x1 de:

$$\mathbf{X} = \left[\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix} \right] \quad \text{con} \quad \mathbf{W} = [\ [[1],[1]]\ ]$$

Solución: $\mathbf{Y} = \mathbf{W} \circledast \mathbf{X} = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 1 & 2 \end{bmatrix}$

In [ ]:
```
W = torch.nn.Conv2d(2, 1, 1, bias=False)
W.weight.data = torch.tensor([ [ [[1]], [[1]] ] ], requires_grad=True, dtype=torch.float)
X = torch.tensor([ [ [[1,0,0],[1,1,1],[1,1,1]], [[1,1,1],[0,0,1],[0,0,1]] ] ], dtype=torch.float)
print(str(W(X).data).replace('\n',''))
```

```
tensor([[[[2., 1., 1.],          [1., 1., 2.],          [1., 1., 2.]]]])
```

4 Pooling

Problema: Halla el maxpooling de $\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$ con un tamaño de filtro $f_h \times f_w = (2, 2)$

Solución: $\mathbf{Y} = \mathrm{MaxPool2d}(\mathbf{X}) = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$

In [ ]:
```
import numpy as np; import torch; P = torch.nn.MaxPool2d(2, stride=1)
X = torch.tensor([[ [[1,1,1], [0,0,1], [0,0,1]] ]], dtype=torch.float)
print(str(P(X).data).replace('\n','').replace(' ',''))
```

```
tensor([[[[1.,1.],[0.,1.]]]])
```

Problema: Halla el avgpooling de $\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$ con un tamaño de filtro $f_h \times f_w = (2, 2)$

Solución: $\mathbf{Y} = \mathrm{AvgPool2d}(\mathbf{X}) = \begin{bmatrix} 0.5 & 0.75 \\ 0 & 0.5 \end{bmatrix}$

In [ ]:
```
P = torch.nn.AvgPool2d(2, stride=1); X = torch.tensor([[ [[1,1,1], [0,0,1], [0,0,1]] ]], dtype=torch.float)
print(str(P(X).data).replace('\n','').replace(' ',''))
```

```
tensor([[[[0.5000,0.7500],[0.0000,0.5000]]]])
```

Problema: Halla el global maxpooling de $\mathbf{X} = \begin{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \end{bmatrix}$ con un tamaño de filtro $f_h \times f_w = (2, 1)$

Solución: $\mathrm{MaxPool2d}(\mathbf{X}) = \begin{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \end{bmatrix}$ $\quad \mathbf{Y} = \mathrm{GlobMaxPool2d}(\mathbf{X}) = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$

In [ ]:
```
P1 = torch.nn.MaxPool2d((2, 1), stride=1);
X = torch.tensor([[ [[0], [1], [2]], [[1], [1], [1]] ]], dtype=torch.float)
P1X = P1(X); print('MaxPool2d =', P1X.shape, str(P1X.data).replace('\n','').replace(' ',''))
Y, _ = P1X.squeeze(3).max(2); print('GlobMaxPool2d =', Y)
```

```
MaxPool2d = torch.Size([1, 2, 2, 1]) tensor([[[[1.],[2.]],[[1.],[1.]]]])
GlobMaxPool2d = tensor([[2., 1.]])
```

Problema: Halla el global avgpooling de $\mathbf{X} = \begin{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \end{bmatrix}$ con un tamaño de filtro $f_h \times f_w = (2, 1)$

Solución: $\mathrm{AvgPool2d}(\mathbf{X}) = \begin{bmatrix} \begin{bmatrix} 0.5 \\ 1.5 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \end{bmatrix}$ $\quad \mathbf{Y} = \mathrm{GlobAvgPool2d}(\mathbf{X}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

In [ ]:
```
P1 = torch.nn.AvgPool2d((2, 1), stride=1); P2 = torch.nn.AdaptiveAvgPool2d(1)
X = torch.tensor([[ [[0], [1], [2]], [[1], [1], [1]] ]], dtype=torch.float)
P1X = P1(X); print('AvgPool2d =', P1X.shape, str(P1X.data).replace('\n','').replace(' ',''))
Y = P2(P1X); print('GlobAvgPool2d =', Y.shape, str(Y.data).replace('\n','').replace(' ',''))
```

```
AvgPool2d = torch.Size([1, 2, 2, 1]) tensor([[[[0.5000],[1.5000]],[[1.0000],[1.0000]]]])
GlobAvgPool2d = torch.Size([1, 2, 1, 1]) tensor([[[[1.]],[[1.]]]])
```

5 ImageNet

Cuestión. ImageNet es un dataset muy popular desde los 2010. En relación con el mismo, indica la opción incorrecta o escoge la última opción si las tres primeras son correctas:

1. Recoge más de 14 millones de imágenes etiquetadas con cerca de 22 mil synsets.
2. ImageNet-1K es un subconjunto comúnmente utilizado para clasificación de imágenes que incluye algo más de 1.4M imágenes de 1000 clases.
3. La partición estándar de ImageNet-1K consta de 1.3M imágenes de entrenamiento aproximadamente, 50K de validación y 100K de test; las etiquetas de test no se hallan públicamente disponibles.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

6 AlexNet

Cuestión. AlexNet es una red convolucional (CNN) muy popular, introducida en 2012. En relación con la misma, indica la opción incorrecta o escoge la última opción si las tres primeras son correctas:

1. Se diseñó para participar en el ImageNet Large Scale Visual Recognition Challenge (ILSVRC) de 2012. Los excelentes resultados obtenidos en dicho challenge atrajeron gran interés; de hecho, se considera que inició la revolución del aprendizaje profundo.
2. Consta de cinco capas convolucionales, tres de agrupación y tres lineales.
3. Tanto las capas convolucionales como las lineales, salvo la última, incluyen una función de activación ReLU (en lugar de la sigmoide empleada en LeNet). Otro aspecto novedoso es el uso de dropout.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

7 Dropout

Problema: aplica dropout a $\mathbf{X} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$ en los siguientes casos

1. $p = 0.1,$ asumiendo que no anula ninguna entrada
2. $p = 0.5,$ asumiendo que anula la primera entrada

Solución:

$$\mathrm{Dropout}(\mathbf{X}, 0.1) = \begin{pmatrix} 3 \\ 2 \end{pmatrix} \frac{1}{0.9} = \begin{pmatrix} 3.3333 \\ 2.2222 \end{pmatrix}$$

$$\mathrm{Dropout}(\mathbf{X}, 0.5) = \begin{pmatrix} 0 \\ 2 \end{pmatrix} \frac{1}{0.5} = \begin{pmatrix} 0 \\ 4 \end{pmatrix}$$

In [ ]:
```
import torch; torch.random.manual_seed(23); X = torch.tensor([3, 2], dtype=torch.float)
print(torch.nn.Dropout(p=0.1)(X), torch.nn.Dropout(p=0.5)(X))
```

```
tensor([3.3333, 2.2222]) tensor([0., 4.])
```

8 Inception

Cuestión. Inception es una red convolucional (CNN) propuesta por Google en 2014. Introduce la meta-arquitectura típica de las CNNs modernas: stem (raíz), body (cuerpo) y head (cabeza). En relación con esta meta-arquitectura, indica la opción incorrecta o escoge la última opción si las tres primeras son correctas:

1. La raíz se compone de dos o tres capas convolucionales que extraen características de bajo nivel.
2. El cuerpo constituye una subred de bloques convolucionales repetidos. El bloque inception emplea múltiples caminos paralelos, cada uno de los cuales aplica un filtrado convolucional distinto.
3. La cabeza transforma la salida del cuerpo según la tarea a abordar (clasificación, segmentación, etc.).
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

9 ResNet

Cuestión. ResNet es una red convolucional (CNN) desarrollada por Microsoft en 2015. Introduce conexiones residuales para entrenar modelos muy profundos. En relación con la misma, indica la opción incorrecta o escoge la última opción si las tres primeras son correctas:

1. La raíz consta de una convolución 7×7, seguida de batch norm y maxpooling 3×3.
2. El cuerpo se compone de bloques residuales sucesivos. Cada bloque residual procesa su entrada mediante dos sub-bloques "convolución 3×3, batch norm y ReLU". La conexión residual consiste en añadir una conexión aditiva de la entrada al bloque y la pre-activación del segundo sub-bloque. De esta manera, el bloque residual puede verse como una copia simple de la entrada a la que se suma una transformación residual.
3. La cabeza se reduce a una sola capa lineal alimentada por el global avgpooling de la salida del cuerpo.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

10 Capas de normalización

Cuestión. Las capas de normalización batch, layer, instance y group transforman el batch de entrada de manera análoga, mediante estandarización con:

$$y = \frac{x - \mathrm{E}[x]}{\sqrt{\mathrm{Var}[x] + \epsilon}}\,\gamma + \beta$$

donde $\gamma = 1$ y $\beta = 0$ pueden aprenderse; $\epsilon = 10^{-5}$ por omisión. La principal diferencia entre ellas reside en el modo en el que procesan la entrada para estimar los estadísticos necesarios, esto es, medias y varianzas. Supón que la entrada es un 4-tensor $N \times C \times H \times W$ correspondiente a N imágenes o mapas de características de C canales, altura H y anchura W. En relación con estas capas de normalización, indica la opción incorrecta o escoge la última opción si las tres primeras son correctas:

1. Batch opera canal a canal; por cada canal, calcula estadísticos sobre el resto de dimensiones.
2. Layer procesa cada dato separadamente; por cada dato, calcula estadísticos sobre el resto de dimensiones.
3. Instance depende de canal y dato; por cada par canal-dato, calcula estadísticos sobre altura y anchura.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

Problema: Halla $\mathbf{Y} = \mathrm{BatchNorm2d}(\mathbf{X})$ con $\mathbf{X} = [[[2],[3]],[[1],[1]]]$

Solución: $\mathbf{Y} = [[[1],[1]],[[-1],[-1]]]$

In [ ]:
```
import torch; X = torch.tensor([ [ [[2]], [[3]] ], [ [[1]], [[1]] ] ], dtype=float)
print(X.shape, str(X).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 2, 1, 1]) tensor([[[[2.]],[[3.]]],[[[1.]],[[1.]]]],dtype=torch.float64)
```

In [ ]:
```
Y = torch.nn.BatchNorm2d(num_features=2, eps=0., momentum=0., affine=False, dtype=X.dtype)(X)
print(Y.shape, str(Y).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 2, 1, 1]) tensor([[[[1.]],[[1.]]],[[[-1.]],[[-1.]]]],dtype=torch.float64)
```

Paso a paso:

In [ ]:
```
X_flat = X.transpose(0, 1).flatten(start_dim=1)
print(X_flat.shape, str(X_flat).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 2]) tensor([[2.,1.],[3.,1.]],dtype=torch.float64)
```

In [ ]:
```
X_mean = X_flat.mean(dim=1).reshape(-1, 1)
print(X_mean.shape, str(X_mean).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 1]) tensor([[1.5000],[2.0000]],dtype=torch.float64)
```

In [ ]:
```
X_std = X_flat.std(dim=1, correction=0).reshape(-1, 1)
print(X_std.shape, str(X_std).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 1]) tensor([[0.5000],[1.0000]],dtype=torch.float64)
```

In [ ]:
```
Y_alt = ((X_flat - X_mean) / X_std).transpose(0, 1).reshape(X.shape); (Y - Y_alt).abs().max().item()
```

Out[ ]:
```
0.0
```

Problema: Halla $\mathbf{Y} = \mathrm{LayerNorm}(\mathbf{X})$ con $\mathbf{X} = \left[\left[\begin{bmatrix}2\\1\end{bmatrix}, \begin{bmatrix}1\\2\end{bmatrix}\right]\right]$

Solución: $\mathbf{Y} = \left[\left[\begin{bmatrix}1\\-1\end{bmatrix}, \begin{bmatrix}-1\\1\end{bmatrix}\right]\right]$

In [ ]:
```
import torch; X = torch.tensor([ [ [[2], [1]], [[1], [2]] ] ], dtype=float)
print(X.shape, str(X).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 2, 2, 1]) tensor([[[[2.],[1.]],[[1.],[2.]]]],dtype=torch.float64)
```

In [ ]:
```
Y = torch.nn.LayerNorm(X.shape[1:], eps=0., elementwise_affine=False, bias=False, dtype=X.dtype)(X)
print(Y.shape, str(Y).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 2, 2, 1]) tensor([[[[1.],[-1.]],[[-1.],[1.]]]],dtype=torch.float64)
```

Paso a paso:

In [ ]:
```
X_flat = X.flatten(start_dim=1)
print(X_flat.shape, str(X_flat).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 4]) tensor([[2.,1.,1.,2.]],dtype=torch.float64)
```

In [ ]:
```
X_mean = X_flat.mean(dim=1).reshape(-1, 1)
print(X_mean.shape, str(X_mean).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1]) tensor([[1.5000]],dtype=torch.float64)
```

In [ ]:
```
X_std = X_flat.std(dim=1, correction=0).reshape(-1, 1)
print(X_std.shape, str(X_std).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1]) tensor([[0.5000]],dtype=torch.float64)
```

In [ ]:
```
Y_alt = ((X_flat - X_mean) / X_std).reshape(X.shape); (Y - Y_alt).abs().max().item()
```

Out[ ]:
```
0.0
```

Problema: Halla $\mathbf{Y} = \mathrm{InstanceNorm2d}(\mathbf{X})$ con $\mathbf{X} = \begin{bmatrix} \begin{bmatrix} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \end{bmatrix} \end{bmatrix}$

Solución: $\mathbf{Y} = \begin{bmatrix} \begin{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \end{bmatrix} \end{bmatrix}$

In [ ]:
```
import torch; X = torch.tensor([ [ [ [2, 1], [1, 2] ] ] ], dtype=float)
print(X.shape, str(X).replace('\n',''))
```

```
torch.Size([1, 1, 2, 2]) tensor([[[[2., 1.],          [1., 2.]]]], dtype=torch.float64)
```

In [ ]:
```
Y = torch.nn.InstanceNorm2d(num_features=1, eps=0., momentum=0., affine=False, dtype=X.dtype)(X)
print(Y.shape, str(Y).replace('\n',''))
```

```
torch.Size([1, 1, 2, 2]) tensor([[[[ 1., -1.],          [-1.,  1.]]]], dtype=torch.float64)
```

Paso a paso:

In [ ]:
```
X_flat = X.flatten(start_dim=2)
print(X_flat.shape, str(X_flat).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1, 4]) tensor([[[2.,1.,1.,2.]]],dtype=torch.float64)
```

In [ ]:
```
X_mean = X_flat.mean(dim=2).reshape(-1, 1)
print(X_mean.shape, str(X_mean).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1]) tensor([[1.5000]],dtype=torch.float64)
```

In [ ]:
```
X_std = X_flat.std(dim=2, correction=0).reshape(-1, 1)
print(X_std.shape, str(X_std).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1]) tensor([[0.5000]],dtype=torch.float64)
```

In [ ]:
```
Y_alt = ((X_flat - X_mean) / X_std).reshape(X.shape); (Y - Y_alt).abs().max().item()
```

Out[ ]:
```
0.0
```

Problema: Halla $\mathbf{Y} = \mathrm{GroupNorm}(\mathbf{X}, 2)$ con $\mathbf{X} = [[[2],[1],[3],[1]]]$

Solución: $\mathbf{Y} = [[[1],[-1],[1],[-1]]]$

In [ ]:
```
import torch; X = torch.tensor([ [ [[2]], [[1]], [[3]], [[1]] ] ], dtype=float)
print(X.shape, str(X).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 4, 1, 1]) tensor([[[[2.]],[[1.]],[[3.]],[[1.]]]],dtype=torch.float64)
```

In [ ]:
```
Y = torch.nn.GroupNorm(num_groups=2, num_channels=4, eps=0., affine=False, dtype=X.dtype)(X)
print(Y.shape, str(Y).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 4, 1, 1]) tensor([[[[1.]],[[-1.]],[[1.]],[[-1.]]]],dtype=torch.float64)
```

Paso a paso:

In [ ]:
```
X_flat = X.reshape(X.shape[0], 2, X.shape[1]//2, *X.shape[2:]).flatten(start_dim=2)
print(X_flat.shape, str(X_flat).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 2, 2]) tensor([[[2.,1.],[3.,1.]]],dtype=torch.float64)
```

In [ ]:
```
X_mean = X_flat.mean(dim=2).reshape(-1, 1)
print(X_mean.shape, str(X_mean).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 1]) tensor([[1.5000],[2.0000]],dtype=torch.float64)
```

In [ ]:
```
X_std = X_flat.std(dim=2, correction=0).reshape(-1, 1)
print(X_std.shape, str(X_std).replace('\n','').replace(' ',''))
```

```
torch.Size([2, 1]) tensor([[0.5000],[1.0000]],dtype=torch.float64)
```

In [ ]:
```
Y_alt = ((X_flat - X_mean) / X_std).reshape(X.shape); (Y - Y_alt).abs().max().item()
```

Out[ ]:
```
0.0
```

Problema: Halla $\mathbf{Y} = \mathrm{RMSNorm}(\mathbf{X}, (2,2))$ con $\mathbf{X} = \begin{bmatrix} \begin{bmatrix} \begin{bmatrix} 3 & 1 \\ 5 & 1 \end{bmatrix} \end{bmatrix} \end{bmatrix}$

Solución: $\mathbf{Y} = \begin{bmatrix} \begin{bmatrix} \begin{bmatrix} 1 & 1/3 \\ 5/3 & 1/3 \end{bmatrix} \end{bmatrix} \end{bmatrix}$

In [ ]:
```
import torch; X = torch.tensor([ [ [ [3, 1], [5, 1] ] ] ], dtype=float)
print(X.shape, str(X).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1, 2, 2]) tensor([[[[3.,1.],[5.,1.]]]],dtype=torch.float64)
```

In [ ]:
```
Y = torch.nn.RMSNorm(normalized_shape=X.shape[-2:], eps=0., elementwise_affine=False, dtype=X.dtype)(X)
print(Y.shape, str(Y).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1, 2, 2]) tensor([[[[1.0000,0.3333],[1.6667,0.3333]]]],dtype=torch.float64)
```

Paso a paso:

In [ ]:
```
X_flat = X.flatten(start_dim=-2)
print(X_flat.shape, str(X_flat).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1, 4]) tensor([[[3.,1.,5.,1.]]],dtype=torch.float64)
```

In [ ]:
```
X_rms = X_flat.square().mean(dim=-1).reshape(-1, 1).sqrt()
print(X_rms.shape, str(X_rms).replace('\n','').replace(' ',''))
```

```
torch.Size([1, 1]) tensor([[3.]],dtype=torch.float64)
```

In [ ]:
```
Y_alt = (X_flat / X_rms).reshape(X.shape); (Y - Y_alt).abs().max().item()
```

Out[ ]:
```
2.220446049250313e-16
```

11 Transformer

Cuestión. Transformer es una arquitectura propuesta por Google en 2017 para traducción automática. En relación con la misma, indica la opción incorrecta o escoge la última opción si las tres primeras son correctas:

1. Su principal novedad es que se basa solamente en mecanismos de atención.
2. El encoder obtiene una representación enriquecida de la frase a traducir.
3. El decoder genera la traducción paso a paso, auto-regresivamente, a partir de la salida del encoder.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.

12 Vision Transformer

Cuestión. Vision Transformer (ViT) es una adaptación del Transformer para visión propuesta en 2020. En relación con la misma, indica la opción incorrecta o escoge la última opción si las tres primeras son correctas:

1. La imagen se divide en patches de tamaño fijo que se transforman linealmente en tokens vectoriales, a los que se suman codificaciones vectoriales de sus posiciones, obteniéndose una secuencia de tokens vectoriales a modo de frase a traducir.
2. Se aplica un encoder Transformer convencional, con pre-normalización en lugar de post-normalización, a la representación secuencial de la imagen con el fin de obtener una representación enriquecida de la misma.
3. El decoder se reduce a una cabeza MLP para clasificación.
4. Las tres afirmaciones anteriores son correctas.

Solución: La 4.